Titanic Classification Practical

Preprocessing

The Titanic dataset was loaded and preprocessed by:
• Handling missing Age and Embarked values
• Removing the Cabin column
• Encoding Sex and Embarked
• Saving the cleaned dataset as cleaned_titanic.csv

Classification Code

```
# CLASSIFICATION MODELS
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

columns_to_drop = ['Survived', 'Name', 'Ticket', 'PassengerId']
X = titanic_data.drop(columns=[c for c in columns_to_drop if c in titanic_data.columns])
y = titanic_data['Survived']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# Logistic Regression
logistic_model = LogisticRegression(max_iter=1000)
logistic_model.fit(X_train, y_train)
y_pred_logistic = logistic_model.predict(X_test)
logistic_accuracy = accuracy_score(y_test, y_pred_logistic)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_logistic)
print("Confusion Matrix:\n", cm)
ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['Did Not Survive', 'Survived']
).plot()
plt.title("Logistic Regression - Confusion Matrix")
plt.show()

# Decision Tree
decision_tree = DecisionTreeClassifier(random_state=42)
decision_tree.fit(X_train, y_train)
y_pred_tree = decision_tree.predict(X_test)
tree_accuracy = accuracy_score(y_test, y_pred_tree)

# KNN
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
y_pred_knn = knn_model.predict(X_test)
knn_accuracy = accuracy_score(y_test, y_pred_knn)

print("Logistic Regression:", round(logistic_accuracy * 100, 2), "%")
print("Decision Tree:", round(tree_accuracy * 100, 2), "%")
print("KNN:", round(knn_accuracy * 100, 2), "%")

# Decision Tree max_depth tuning
depth_values = [2, 3, 4, 5, 6, 8, 10, None]
depth_results = []

for depth in depth_values:
    model = DecisionTreeClassifier(max_depth=depth, random_state=42)
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    acc = accuracy_score(y_test, predictions)
    depth_results.append({
        'max_depth': depth,
        'accuracy (%)': round(acc * 100, 2)
    })

depth_results_df = pd.DataFrame(depth_results)
display(depth_results_df)
```

This practical applies Logistic Regression, Decision Tree, and KNN to predict Titanic passenger survival. Model accuracy, a confusion matrix, model comparison, and Decision Tree max_depth tuning are included.